

MVP blueprint for an AI customer support product for Indian D2C sellers

Market reality

This idea is commercially credible in India ¹ because the market has the three ingredients that matter most for a support-automation startup: very large digital commerce volume, a huge long tail of small merchants, and consumer behavior that still creates expensive manual support work. India had about 1.028 billion internet subscribers by the end of December 2025, the Ministry of MSME dashboard showed roughly 79.4 million registered MSMEs on May 2, 2026, and UPI processed 22.64 billion transactions worth ₹29.53 lakh crore in March 2026. Bain estimated India's e-retail market at about \$60 billion GMV in 2024 with more than 270 million online shoppers, while IIMA reported that nearly 65% of surveyed Indian consumers used cash on delivery in their last online transaction. That combination makes order-status questions, COD confirmation, returns, refunds, and payment trust a very large recurring support surface. ²

The global reference set is already proven. Netomi, Intercom Fin, Gorgias, and Ada all sell some mix of AI resolution, omnichannel support, workflow automation, and handoff. Gorgias is explicitly positioned around ecommerce support and says it is trusted by 17,000 brands; Ada says it can autonomously resolve up to 83% of support issues; Intercom positions Fin as a customer-service AI agent; Netomi positions itself as a full agent lifecycle platform. These companies prove that the category is real, but they are built for broader or higher-maturity support stacks than many small Indian brands have. ³

In the Indian SMB/D2C layer, the current competition is real but still leaves room for a sharper wedge. Interakt ⁴ is a WhatsApp-first growth and support platform and publicly lists plans starting at ₹2,499 per month. DelightChat ⁵ is focused on omnichannel support for ecommerce with public startup pricing from \$29 per month. WATI ⁶ is positioned as an end-to-end WhatsApp API solution for SMBs, and LimeChat ⁷ is increasingly pushing AI-led support and growth for Indian brands. Reuters also reported that LimeChat's bots were handling 70% of customer complaints for some clients and that Indian clients were paying around ₹100,000 per month for higher-end automation. That means your best entry point is **not** "build a generic omnichannel helpdesk." It is a narrower product: **post-purchase AI support autopilot for small Indian Shopify-first D2C brands on WhatsApp and Instagram, with COD confirmation and human handoff built in.** ⁸

Problem definition

The core problem is not "customers ask questions." The real problem is: **small D2C brands have fragmented channels, incomplete live order context, inconsistent policies, high-COD friction, and too little operational discipline to answer fast and correctly at scale.** The AI layer only works if it solves each of those pieces explicitly. Recent research strengthens that conclusion: RAG helps grounding, but long context alone is unreliable; structured agent orchestration outperforms naive prompting; customer-support systems need policy adherence, not just fluent text generation; and multilingual customer-service

benchmarks show translated or clean text can overestimate real-world performance on noisy native queries. ⁹

Minute problem	Why it hurts Indian D2C brands	MVP solution
Channel fragmentation	Customers ask on WhatsApp, Instagram, website chat, and email, but the brand's order truth usually lives elsewhere	One conversation inbox with customer identity merge and one order-context panel
Live order-state gap	Support agents cannot answer "where is my order?" without checking Shopify, courier pages, or Excel sheets	Pull live order and fulfillment context into every thread
Policy ambiguity	Return, refund, exchange, COD, size, and cancellation rules are often buried in messy FAQs	Convert merchant policies into structured Intent-Context-Action rules
COD friction	COD remains important in India; fake, abandoned, and unconfirmed orders inflate RTO and support volume	Add COD confirmation and pre-dispatch validation flows
Language messiness	Customers use English, Hindi, Hinglish, typos, voice-note style phrasing, and code-mixing	Add robust language detection, intent classification, and fallback routing
Weak handoff	Most bots fail when the answer is uncertain or emotionally sensitive	Confidence-based handoff with full transcript and order context
Marketplace fragmentation	Different data surfaces for store, courier, and marketplaces make automation brittle	Start with cleanest integrations first and keep marketplace support read-only in early versions
Compliance and trust	Support touches phone numbers, addresses, order value, refunds, and templates on heavily governed channels	Build explicit consent, logging, audit, retention, and operator override from day one
Hallucination risk	A fluent wrong answer in support is worse than no answer	Ground every answer in merchant KB + live tools; never answer from public web at runtime

That table is not theory; it is the practical synthesis of Indian commerce behavior, current channel rules, and support-research findings. Meta's channel model enforces platform-specific rules such as the 24-hour service window and message-template categories, Shopify exposes live order/return data through APIs and webhooks, Amazon and Flipkart require platform-specific seller integrations and access controls, and current customer-support research shows policy-aware orchestration and structured knowledge representation outperform unstructured "just use a chatbot" approaches. ¹⁰

The product therefore should be framed very narrowly:

Target user: founder-led or small-team D2C brands doing support mainly on WhatsApp/Instagram, usually running on Shopify ¹¹ or a similar storefront, and feeling pain around WISMO, COD, cancellations, refunds, and repetitive FAQ traffic.

Target end customer jobs: track my order, confirm my COD order, change address before dispatch, ask return/refund status, ask product/size/ingredient questions, and get to a human fast when the request is complex.

Target merchant promise: reduce first-response time, increase auto-resolution on low-risk intents, cut manual agent effort, and reduce COD-related leakage without breaking channel policy or customer trust.

¹²

MVP scope

For the MVP, the right scope is **WhatsApp + Instagram + Shopify**, with optional courier integration if the merchant uses a shipping aggregator. Do not start by promising full native support across Amazon ¹³, Flipkart ¹⁴, Meesho ¹⁵, and ONDC ¹⁶. Amazon has official SP-API orders, notifications, returns reports, and PII access workflows, but buyer information access is controlled and adds approval overhead. Flipkart has official seller APIs for orders, returns, and notifications. ONDC is a protocol with a deeper seller-app integration path. That makes them important later, but not the fastest path to MVP. ¹⁷

The MVP should include these features, and no more:

Inbound support resolution. The bot must read inbound WhatsApp and Instagram messages, identify the customer, classify the intent, retrieve policy snippets, fetch the live order state if available, and answer or escalate. Meta's WhatsApp Business Platform and Instagram Messaging API both support webhook-driven messaging flows, with Instagram requiring a professional account linked appropriately. ¹⁸

The six mandatory support intents.

- WISMO or delivery-status queries.
- COD confirmation and COD cancellation.
- Pre-dispatch address change or cancellation requests.
- Return and refund policy queries, plus self-serve return initiation where the merchant stack allows it.
- Product FAQ such as size, ingredients, usage, stock, bundle contents, and shipping timelines.
- Human-agent handoff.

These are the flows most directly supported by WhatsApp utility/service messaging and Shopify order/return data surfaces. ¹⁹

A lightweight agent console. The merchant team needs a shared inbox, customer profile, order sidebar, tags, notes, assignment, SLA timer, macros, and an audit trail showing why the AI answered the way it did. Without that operator layer, the system becomes impossible to trust or debug. Research on policy-aware customer-support agents and OpenAI's own support-guardrail examples both point to the need for structured policy traces and an eval-heavy human-review loop. ²⁰

Merchant knowledge ingestion. The AI must ingest only merchant-approved knowledge for runtime answering: store FAQ, shipping policy, return/refund policy, cancellation rules, COD policy, size charts, product catalog, and brand voice. Shopify's APIs and webhooks are the cleanest place to start for order and

return context, while market research and research papers belong in an internal research corpus, not in the customer-facing runtime knowledge base. ²¹

WhatsApp policy-aware outbound flows. The system must manage the difference between free-form service messages inside the customer-service window and templated utility/authentication/marketing messages outside it. Meta explicitly distinguishes these categories and charges on delivered templated messages by category and market; utility templates delivered within an open customer-service window are free, and non-template messages are free-form only within the allowed service window. That means your workflow engine needs a message-category state machine, not just “send message.” ²²

Onboarding must be productized. The MVP must ship with a merchant onboarding checklist: connect the WhatsApp number, verify the business, create/approve templates, connect Instagram, install the Shopify app, grant the minimum viable scopes, subscribe to orders/fulfillment/returns webhooks, upload policy docs, label product FAQs, and configure escalation rules. If onboarding takes a week of manual services work, the unit economics will break. ²³

Just as important are the non-goals. The MVP should **not** be a general-purpose AI assistant on WhatsApp, because Meta’s policy update effective January 15, 2026 restricts general-purpose AI assistants to Meta’s dedicated surfaces and still permits business-purpose support/order-processing use cases on the Business Platform. The MVP should also **not** promise autonomous refunds, credit issuance, or free-form internet answers. It should stay business-specific, policy-grounded, and low-agency. ²⁴

Data and system design

The first architectural mistake to avoid is indiscriminate “scrape the whole internet.” The correct approach is a **governed two-layer corpus**. Layer one is an internal research corpus for you and your team: platform docs, pricing pages, research papers, security standards, and Indian market references. Layer two is the runtime merchant knowledge base that the bot is allowed to cite when answering customers: merchant policies, product data, return rules, shipping rules, FAQs, and live order tools. RAG helps grounded generation, but RAG alone does not eliminate hallucination; long context can fail in the middle of large prompts; and the best support-agent results now come from structured policies plus tool use, not from “stuff everything into context.” ²⁵

The lowest-risk MVP architecture is an event-driven system with five services:

- **Channel gateway:** receives Meta webhooks for WhatsApp and Instagram, verifies signatures, normalizes inbound events, and sends outbound replies. ²⁶
- **Commerce connector:** connects to Shopify APIs and webhooks for orders, fulfillment, returns, and product data. ²⁷
- **Support brain:** runs intent detection, retrieval, policy selection, answer generation, and confidence scoring. It should use hybrid retrieval plus tool results, not public-web answering. ²⁸
- **Inbox and operator app:** shared thread view, assignment, handoff, notes, macros, and audit logs. ²⁹
- **Analytics and eval service:** tracks automation rate, containment, policy adherence, hallucination failures, response latency, and template performance. ³⁰

Two runtime design decisions are non-negotiable. First, **all webhook processing must be idempotent** because Shopify can deliver duplicate webhooks and retries failed deliveries; Meta also retries undelivered webhooks. Second, the system should decouple ingestion from resolution with a queue so the webhook endpoint responds immediately and processing happens asynchronously. ³¹

A good MVP algorithm stack is simple, not exotic:

- **Language and script routing:** detect English, Hindi, and Hinglish/code-mixed patterns; normalize common transliteration variants. Real customer-service benchmarks show noisy native queries are meaningfully harder than translated ones. ³²
- **Hierarchical intent classification:** first predict a broad class such as order status, COD, returns/refunds, product question, cancellation, or human handoff; then predict a sub-intent. This mirrors recent multilingual support benchmarks and reduces downstream complexity. ³³
- **Hybrid retrieval:** combine lexical relevance, semantic similarity, document authority, and recency. Long-context-only prompting is less reliable than targeted retrieval. ³⁴
- **Policy graph execution:** translate messy policy text into structured “Intent, Context, Action” units and route the agent through them. That is directly supported by the Airbnb customer-support paper and the JourneyBench results on policy adherence. ³⁵
- **Confidence-based handoff:** if intent confidence is low, policy support is weak, sentiment is negative, or a high-risk action is requested, escalate. OpenAI’s support-guardrail examples and recent hallucination-eval work both point in this direction. ³⁶

For the actual scoring logic, keep the math understandable. A practical MVP can use formulas like:

- **Intent confidence** = $\max(\text{classifier probability}, \text{similarity to labeled exemplars})$.
- **Retrieval score** = $0.40 * \text{lexical} + 0.40 * \text{semantic} + 0.15 * \text{policy-authority} + 0.05 * \text{recency}$.
- **Priority score** = $\text{SLA weight} + \text{negative sentiment bonus} + \text{high-order-value bonus} + \text{shipping-stage urgency bonus}$.
- **Escalation rule** = escalate if $\text{confidence} < \text{threshold}$, or requested action is risky, or no grounded snippet exists, or customer explicitly asks for a human.

That is more than enough for MVP. Do **not** start with custom model fine-tuning. Start with strong prompting, clear retrieval, merchant-specific rules, and a hard eval harness. ³⁷

The minimum runtime data model should include: `Merchant`, `ChannelConnection`, `Customer`, `Conversation`, `Message`, `Order`, `Fulfillment`, `ReturnRequest`, `KnowledgeDocument`, `PolicyRule`, `Template`, `EscalationRule`, `Assignment`, `AuditLog`, `ConsentRecord`, and `EvalCase`. That gives you enough structure to support search, traceability, analytics, and later expansion without building a monolith too early. ³⁸

Security and compliance

Your MVP is touching phone numbers, addresses, order IDs, conversation history, refund events, and sometimes payment-adjacent data. That means security and compliance are not “enterprise features later”; they are part of the product definition from the first pilot. India’s DPDP framework was operationalized with the DPDP Rules notified on November 14, 2025. PIB’s summary says the framework reinforces rights to

access, correct, update, or erase personal data, and Reuters reported that the rules require companies to limit personal data collection to what is necessary for a defined purpose and to inform individuals in the event of a data breach. ³⁹

For your product, that translates into very concrete controls:

- collect only the fields needed for support resolution and audit;
- separate merchant-specific data from shared product telemetry;
- give merchants deletion/export controls;
- let end users opt out of non-essential messaging;
- store retention policies explicitly;
- never use one merchant's conversations to answer another merchant's user unless the data is fully de-identified and contractually permitted. ⁴⁰

CERT-In remains relevant too. Its 2022 directions require reporting certain cyber incidents within six hours, and the directions also require service providers, intermediaries, data centres, body corporates, and government organisations to enable logs of ICT systems and maintain them securely for a rolling period of 180 days within Indian jurisdiction. The official FAQ clarifies the six-hour reporting expectation and the continuing confidentiality obligations around customer data. ⁴¹

On the AI-security side, use the NIST Generative AI Profile and the OWASP Top 10 for LLM Applications as your baseline security checklist. NIST positions the GenAI profile as a companion resource for governing, mapping, measuring, and managing GenAI risks. OWASP highlights prompt injection, insecure output handling, sensitive information disclosure, excessive agency, and supply-chain vulnerabilities as core LLM risks. For this product, the most relevant mitigations are scoped tools, response grounding, output validation, strict action allowlists, prompt-injection defenses in retrieved documents, and human approval for any state-changing action. ⁴²

The minimal trust architecture for the MVP should therefore include: HMAC or signature verification on inbound webhooks, idempotency keys, role-based access control, encrypted secrets, PII masking in logs, tenant isolation, audit trails for every AI decision, safe-mode fallbacks when confidence is low, and a “human can always take over” control. Shopify's HMAC guidance, Flipkart's signature-based notification docs, and Meta's webhook model all support this design direction. ⁴³

Agent operating model and build prompts

Because you want to build this with codex / antigravity / cursor / opencode, the best setup is **one shared project instruction layer plus many specialist sub-agents**. That matches how current coding-agent platforms are evolving: Codex supports skills and parallel work surfaces, Cursor supports persistent project rules and emphasizes verifiable goals and tests, OpenCode supports `AGENTS.md`, subagents, and on-demand skills, and Antigravity is explicitly agent-first with a mission-control style workflow. ⁴⁴

Your multi-agent roster should look like this:

Agent	Primary job	Required outputs
CEO agent	Strategy, ICP, wedge, KPI tree	STRATEGY.md , ICP.md , KPI_TREE.md
Product manager agent	PRD, roadmap, user stories, acceptance criteria	PRD.md , ROADMAP.md , USER_STORIES.md
Business analyst agent	workflow mapping, merchant onboarding, requirements traceability	BRD.md , RTM.md , ONBOARDING_CHECKLIST.md
Support operations agent	SOPs, macros, escalation rules, agent playbook	SOPS.md , MACROS.md , ESCALATION.md
Data analyst agent	KPI dashboard, cohort metrics, funnel definitions	METRICS_SPEC.md , DASHBOARD_QUERIES.sql
Data scientist agent	intent taxonomy, confidence logic, eval sets	INTENT_TAXONOMY.md , EVAL_PLAN.md , CONFIDENCE.md
UI/UX designer agent	user flows, wireframes, component states	UX_FLOWS.md , WIREFRAME_NOTES.md , COMPONENT_STATES.md
Web designer agent	landing page, onboarding flow, visual system	LANDING_PAGE_COPY.md , DESIGN_TOKENS.md
Supply chain analyst agent	COD, shipping, NDR, tracking, return-state logic	LOGISTICS_RULES.md , STATUS_MAP.md
Finance manager agent	pricing, margin model, BSP cost model, hosting budget	PRICING.md , UNIT_ECONOMICS.xlsx , BUDGET.md
Sales and marketing agent	GTM, demo scripts, objection handling, outreach	GTM.md , DEMO_SCRIPT.md , OBJECTIONS.md
Security/compliance agent	DPDP, CERT-In, audit controls, threat model	SECURITY.md , THREAT_MODEL.md , RETENTION_POLICY.md
Engineering manager agent	backlog slicing, code-review policy, release plan	ENGINEERING_PLAN.md , RELEASE_CHECKLIST.md
Backend architect agent	APIs, schema, queues, tool orchestration	ARCHITECTURE.md , OPENAPI.yaml , SCHEMA.sql
Frontend lead agent	inbox UI, onboarding, analytics console	FRONTEND_PLAN.md , COMPONENTS.md
QA/release agent	integration tests, policy tests, regression suites	TEST_PLAN.md , SMOKE_TESTS.md , EVAL_CASES.jsonl

Below is the **master build prompt** I would use as the repository-level prompt for Codex, Cursor, OpenCode, or Antigravity:

You are the principal product+engineering squad for a SaaS startup building an AI customer support platform for small Indian D2C brands.

Build an MVP called "SupportCopilot" with these constraints:

Business context:

- ICP: small Indian D2C brands doing customer support mainly on WhatsApp and Instagram, often using Shopify.
- Core merchant pain: messy support across channels, slow responses, repeated order-status questions, COD confirmation issues, return/refund confusion, and weak human handoff.
- Core end-customer intents for v1:
 - 1) where-is-my-order
 - 2) COD confirmation / cancellation
 - 3) pre-dispatch cancellation or address change
 - 4) return/refund policy and return request initiation
 - 5) product FAQ
 - 6) talk to a human
- Do not build a general-purpose chatbot.
- Do not answer from the public internet at runtime.
- Only answer from merchant-provided knowledge, merchant policies, product catalog, and live order tools.

Channel and policy rules:

- WhatsApp and Instagram are the only messaging channels in v1.
- WhatsApp must support the distinction between:
 - in-window free-form service replies
 - out-of-window template messaging
 - utility templates for order and support updates
- Human handoff must always be available.
- Use low-agency automation:
 - AI may answer, classify, summarize, suggest, and draft
 - AI may trigger only low-risk workflows by default
 - high-risk workflows require human approval

Commerce and data rules:

- Primary commerce integration is Shopify.
- Design the system so Amazon/Flipkart can be added later, but do not make them required for v1.
- Runtime knowledge base contains only merchant-specific data.
- Research corpus is separate and never directly used to answer end users.

Technical constraints:

- Use a typed codebase.
- Build a production-quality monorepo with:
 - web app
 - backend API

- worker service
- database schema
- queue processing
- tests
- seed data
- local dev setup
- Favor clean modular architecture over cleverness.
- Every external event must be idempotent.
- Every AI response must include traceable grounding metadata internally.
- Every risky action must have policy and audit hooks.

Recommended stack:

- Next.js + TypeScript for frontend
- NestJS or equivalent typed backend
- PostgreSQL
- Redis / job queue
- object storage abstraction for docs
- OpenAPI spec
- test suite for unit, integration, and policy-eval tests

Required features:

1. Merchant onboarding
 - connect WhatsApp
 - connect Instagram
 - connect Shopify
 - upload policies and help docs
 - define escalation rules
2. Unified inbox
 - conversation list
 - conversation detail
 - customer profile
 - live order sidebar
 - AI draft / AI answer / human takeover
3. AI support engine
 - intent classification
 - hybrid retrieval
 - policy rule selection
 - answer generation
 - confidence scoring
 - escalation decision
4. Shopify support tools
 - fetch order
 - fetch fulfillment status
 - fetch returns context
 - configurable low-risk actions
5. Analytics
 - response time
 - automation rate

- handoff rate
- containment rate
- intent distribution
- policy failure count
- 6. Trust and compliance
 - audit logs
 - tenant isolation
 - PII masking in logs
 - retention settings
 - export/delete workflow stubs
- 7. Evaluation harness
 - gold test cases for all 6 core intents
 - hallucination checks
 - policy-adherence tests
 - regression suite

Implementation process:

- First create these files before coding:
 - PRD.md
 - ARCHITECTURE.md
 - DATA_MODEL.md
 - SECURITY.md
 - ONBOARDING.md
 - EVAL_PLAN.md
 - ROADMAP.md
- Then generate the folder structure.
- Then implement in thin vertical slices:
 - slice 1 = merchant onboarding
 - slice 2 = inbox and threading
 - slice 3 = Shopify read-only order context
 - slice 4 = AI answer pipeline
 - slice 5 = handoff + macros + assignment
 - slice 6 = analytics + audit + evals
- For each slice:
 - write/update tests first
 - implement the minimum viable code
 - run tests
 - update docs
 - produce a short changelog

Non-negotiable quality bar:

- strict TypeScript
- lint passes
- tests pass
- no dead code
- no unscoped secrets
- clear environment variable descriptions
- mocks for Meta and Shopify webhooks

- readable code comments only where necessary
- no fake claims in UI copy
- all merchant-facing defaults tuned for Indian D2C realities:
 - COD on by default if merchant enables it
 - English + Hindi/Hinglish examples in seed data
 - order-status and returns flows visible from day one

Now begin by writing the PRD, the architecture decision record, the data model, and the test plan.

Use that prompt with a shared repo instruction layer. Then create an `AGENTS.md` file and role-specific skill files so each coding agent can be invoked surgically. For Codex and OpenCode, map those to skills; for Cursor, map them to project rules plus lightweight skill documents; for Antigravity, map them to dedicated agents/workspaces. The operational principle should be the same across tools: **small tasks, explicit tests, visible artifacts, and role-separated ownership.** ⁴⁵

Economics and open questions

Your pricing idea of **₹3,000 to ₹15,000 per month** is plausible if you stay tightly scoped. Interakt's public pricing starts at ₹2,499 per month, DelightChat publicly starts at \$29 per month, and the more advanced AI-heavy Indian vendors move much higher. That means the best commercial position is: **"cheaper and simpler than a full helpdesk, but deeper than a basic WhatsApp bot."** The easiest bundle to sell is one WhatsApp number, one Instagram account, one Shopify store, shared inbox, AI resolution for the six core intents, and one monthly analytics report. ⁴⁶

The fastest rollout path is:

- **Pilot wedge:** 5–10 Shopify-first brands in beauty, fashion, supplements, or home categories with 500–5,000 monthly support conversations.
- **Commercial promise:** faster replies, fewer repetitive agent touches, cleaner COD confirmation, and better support consistency.
- **Implementation discipline:** start read-only, then enable low-risk operational actions after you prove answer quality and handoff reliability.
- **Core north-star metrics:** first response time, automation rate, grounded-answer rate, handoff rate, containment rate, merchant save-hours, and COD-confirmation completion rate.
- **Do not sell first:** enterprise omnichannel, voice AI, full marketplace OMS, or autonomous refund operations. ⁴⁷

The most important unresolved questions are narrow and manageable. First, whether you will use Meta Cloud API directly or a BSP layer, because BSP economics and onboarding support differ. Second, how much direct marketplace depth your pilot merchants truly need beyond Shopify and courier tracking. Third, how quickly you want to support languages beyond English/Hindi/Hinglish. Fourth, how much operator tooling your first merchants require before trusting automation. None of those questions changes the core conclusion: **the MVP should be a policy-grounded, Shopify-first, Meta-channel support autopilot for Indian D2C brands—not a giant omnichannel platform and not a generic AI chatbot.** ⁴⁸

- 1 43 <https://shopify.dev/docs/apps/build/webhooks/subscribe/https>
<https://shopify.dev/docs/apps/build/webhooks/subscribe/https>
- 2 The Indian Telecom Services Performance Indicators
https://traf.gov.in/sites/default/files/2026-03/QPIR_03032026_0.pdf?utm_source=chatgpt.com
- 3 Netomi — The only agentic AI platform built for what comes ...
https://www.netomi.com/?utm_source=chatgpt.com
- 4 7 21 27 Webhooks
https://shopify.dev/docs/api/webhooks/latest?utm_source=chatgpt.com
- 5 6 13 28 32 33 <https://arxiv.org/abs/2603.23172>
<https://arxiv.org/abs/2603.23172>
- 8 46 <https://www.interakt.shop/pricing/>
<https://www.interakt.shop/pricing/>
- 9 11 25 34 <https://arxiv.org/abs/2005.11401>
<https://arxiv.org/abs/2005.11401>
- 10 <https://developers.facebook.com/documentation/business-messaging/whatsapp/messages/send-messages>
<https://developers.facebook.com/documentation/business-messaging/whatsapp/messages/send-messages>
- 12 29 47 <https://www.delightchat.io/>
<https://www.delightchat.io/>
- 14 42 <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
<https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-generative-artificial-intelligence>
- 15 24 WhatsApp Business Platform - Meta for Developers
https://developers.facebook.com/documentation/business-messaging/whatsapp/pricing/ai-providers/?utm_source=chatgpt.com
- 16 23 Developer Hub | WhatsApp Business
https://business.whatsapp.com/developers/developer-hub?utm_source=chatgpt.com
- 17 <https://developer-docs.amazon.com/sp-api/docs/orders-api>
<https://developer-docs.amazon.com/sp-api/docs/orders-api>
- 18 Webhooks - Meta for Developers - Facebook
https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/overview/?utm_source=chatgpt.com
- 19 WhatsApp Utility Messages | Automate customer notifications
https://business.whatsapp.com/products/conversation-categories/utility?utm_source=chatgpt.com
- 20 30 37 <https://arxiv.org/abs/2601.00596>
<https://arxiv.org/abs/2601.00596>
- 22 WhatsApp Business Platform Pricing | WhatsApp API Pricing
https://business.whatsapp.com/products/platform-pricing?utm_source=chatgpt.com
- 26 <https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/overview/>
<https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/overview/>

31 <https://shopify.dev/docs/apps/build/webhooks/ignore-duplicates>

<https://shopify.dev/docs/apps/build/webhooks/ignore-duplicates>

35 <https://aclanthology.org/2025.coling-industry.42/>

<https://aclanthology.org/2025.coling-industry.42/>

36 https://developers.openai.com/cookbook/examples/developing_hallucination_guardrails

https://developers.openai.com/cookbook/examples/developing_hallucination_guardrails

38 <https://shopify.dev/docs/api/admin-graphql/latest/objects/return>

<https://shopify.dev/docs/api/admin-graphql/latest/objects/return>

39 40 <https://www.pib.gov.in/PressReleasePage.aspx?PRID=2190014>

<https://www.pib.gov.in/PressReleasePage.aspx?PRID=2190014>

41 https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf

https://www.cert-in.org.in/PDF/CERT-In_Directions_70B_28.04.2022.pdf

44 <https://openai.com/codex/>

<https://openai.com/codex/>

45 <https://cursor.com/blog/agent-best-practices>

<https://cursor.com/blog/agent-best-practices>

48 <https://business.whatsapp.com/developers/developer-hub>

<https://business.whatsapp.com/developers/developer-hub>